VDK/TLM Creation

Rapid Training

X-2025.06

Module 3: Lab A

Import Existing SystemC TLM Models

© 2025 Synopsys, Inc.

Table of Contents

Lab Overview *2*

Task-1: Create a new TLM Project using the *systemc_import* Template **2**

Task-2: Specify Source Files and Module to be Imported **3**

Task-3: Import the Model **5**

Task-4: Build TLM Creator Model **8**

Task-5: Executing Unit Tests **9**

What We Learnt *9*

Lab Overview

Your task for this lab is to use the TLM Creation perspective of Virtualizer Studio to import an existing SystemC TLM Model into a TLM project. This flow is typically used while creating a VDK using SystemC models, which are already existing, either written manually or as a product of other flows.

You will start with the SystemC source files from the lab data package and import it in your TLM project. The tool extracts the model properties from the C++ source code. Then, you will build the project so that the model can be used for VDK creation.

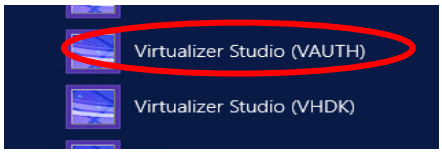
For more details on the overview of this training program and the lab data package, refer to the TLM Creator Training Labs Overview.

Task-1: Create a new TLM Project using the *systemc_import* Template

***Motivation**: In this step you will create a TLMC project using the `systemc_import` template. This template is used to bring existing SystemC models into TLM Creator.*

1. Start Virtualizer Studio

   - Windows: Open Virtualizer Studio (`vze`) from the Synopsys Virtual Prototyping installation using the Windows Startup menu



   - Linux: Open a Console, setup the tool environment, and start Virtualizer Studio in the `module_1` directory as follows:

     - `% source <installDir>/VPProducts/SLS/linux/setup.[sh | csh] -vze`
     - `% cd <TrainingData>/module_3`
     - `% vs&`

   The tool opens and displays the Workspace Launcher window.

2. Specify your workspace location, where you want to save your projects and click ***OK*** on the Workspace Launcher window.

   Note: Workspace is where Virtualizer Studio will store all your model and VDK projects and associated files.

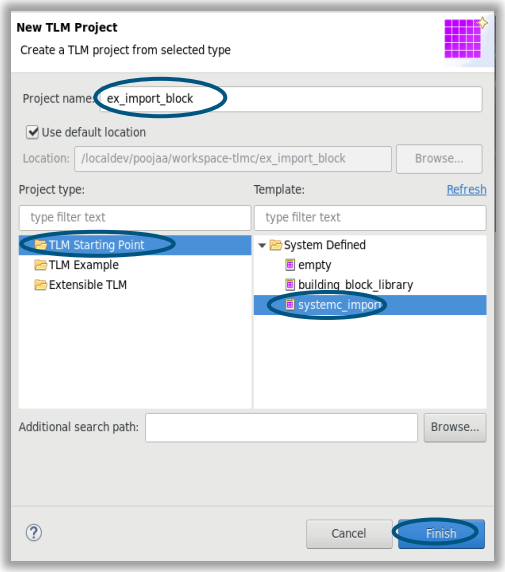
   When the Welcome screen appears, click ***New Project*** under ***TLM Creation***.

3. Specify the following in the ***New TLM Project*** dialog:

   - Type `ex_import_block` in the ***Name*** field.
   - Select `systemc_import` from the ***Model templates***.
   - Click ***Finish***.

   When prompted, click ***Yes*** to open the ***TLM Creation*** perspective.

   The ***TLM Creation*** perspective appears. We will examine the information displayed in different tabs of this perspective in the subsequent sections.

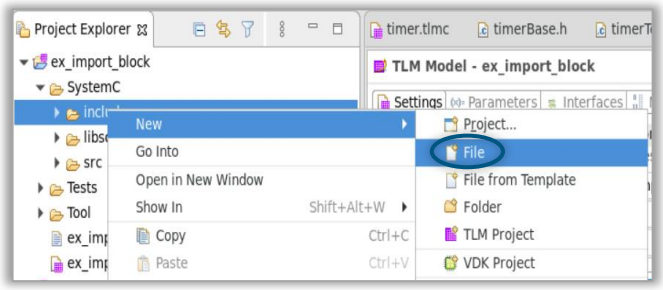


Task-2: Specify Source Files and Module to be Imported

***Motivation**: A SystemC module description is typically coded in header and source files. For importing the model, you need to specify the header and source files. You might also need to specify additionally needed include paths or other settings in the **Build Configurations** so that the module can be successfully parsed.*
You may as well use links to point to the required header or source files. The benefit is that the source files are physically still in the same location, but there are links present in the eclipse workspace so that it is easy to open/edit them from the tool itself.

4. First, create a new file in the TLM project and link it to the provided SystemC header file for the component to be imported using the following steps:

   Right-click on the `include` directory under `SystemC` folder of your project, and select the ***New -> File*** menu option to open the ***New File*** dialog.



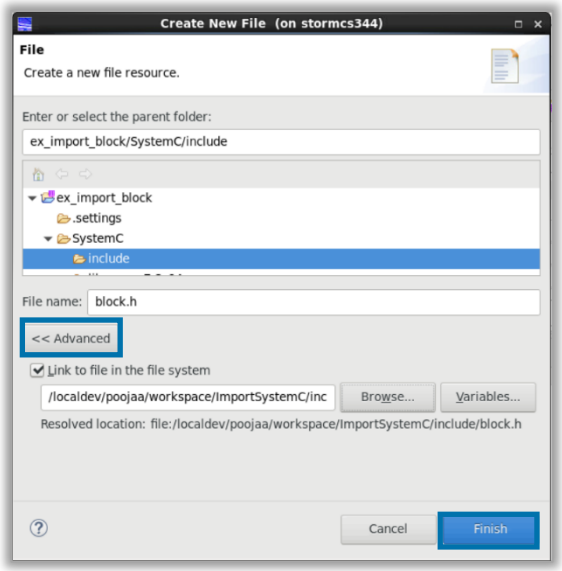
5. In the ***New File*** dialog

   i. Expand ***Advanced*** options.

   ii. Tick the ***Link to file in the file system*** checkbox, and

   iii. Specify the path to the header file, `block.h` from the lab data package as `module_3/ImportSystemC/include` Or use ***Browse…*** to locate the file.

   iv. Click ***Finish***.



   This will create a link to the file. The file can be viewed/edited from the project.

   If more source files are to be added, you can link them in the same way.

   - *Headers* (`.h`) are to be added in the `<project>/SystemC/include` directory.

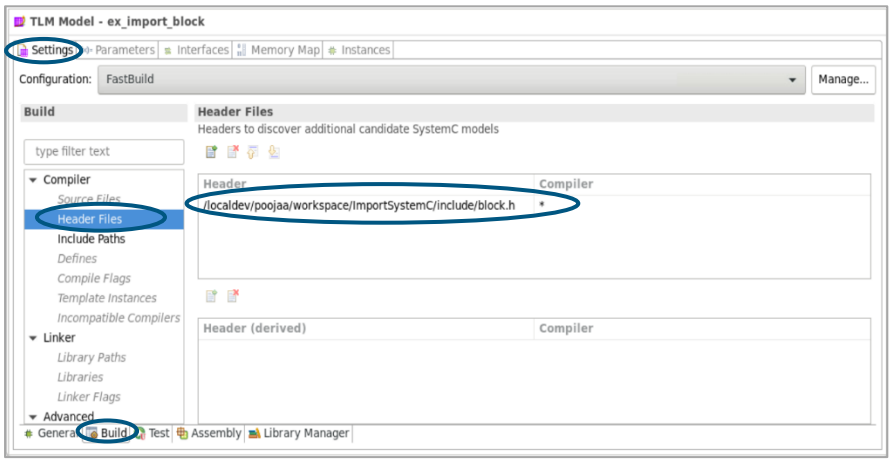
   - *Implementation files* (`.cc/.cpp`) files to be added to `<project>SystemC/src` directory.

   Note that these files are automatically added to the ***Build Configurations*** (both `Release` and `Debug`) of the TLM project.

6. Open the ***Build*** page of the ***Settings*** tab of the Model editor. Open the ***Compiler*** settings and review the ***Header Files*** option.

   The link to the header file (`block.h`) added in the last step, gets reflected in the list of headers automatically.

   You must specify any extra ***Headers***, ***Sources***, ***Include Paths***, and ***Defines*** in the ***Build Configurations*** page.



Task-3: Import the Model

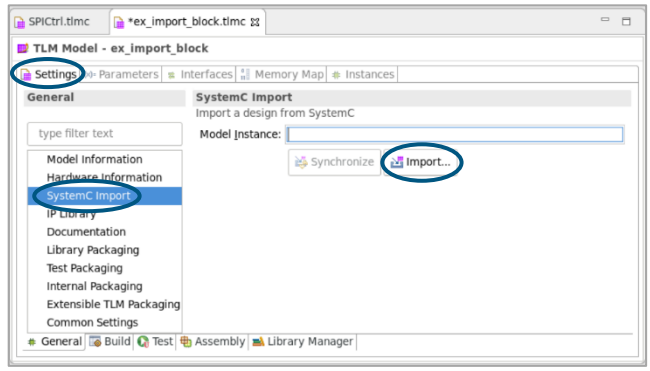
Motivation: Now that we have set up the correct header file of the SystemC model in the TLM Creator project, we can import it.

7. Click to open the ***SystemC Import*** options in the ***General*** page of the ***Settings*** tab of the model editor. Click the ***Import*** button.

   The tool starts to scan the specified files, locates all the modules, and reports errors and messages, if any in the ***Console*** view.

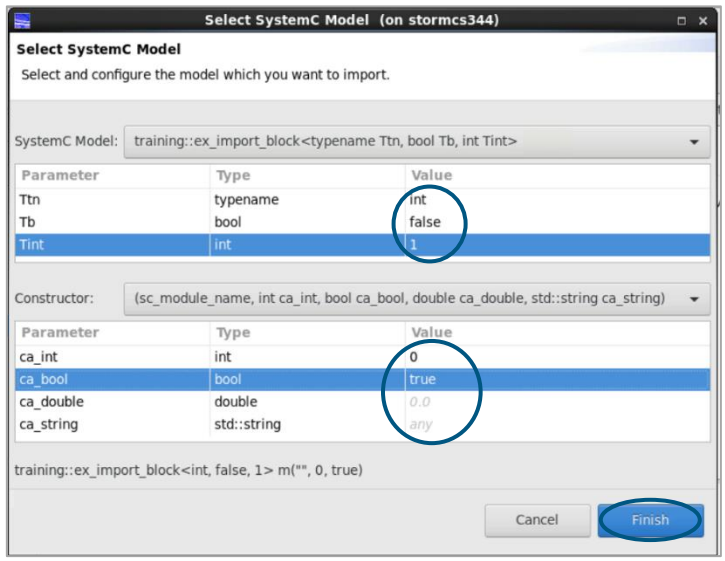
   If needed, fix the errors so that the specified files can be parsed.

   If no errors are reported, the ***Select SystemC Model*** dialog box appears.



8. Specify the following details.

   i. Select the ***SystemC Model*** to be imported from the pull-down list in case multiple models are present in the specified header file and specify the values for *template arguments* that you want to use. For our lab example, specify the following values for the class templates:
      - `int` for *Ttn*,
      - `false` for *Tb*,
      - `1` for *Tint*.



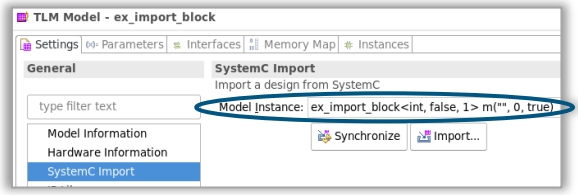
   Note: TLM Creator fully supports C++ templates. During the import process you will need to specify at least a single value set for all template parameters, as this is required for the SystemC importer. Later on, you will be able to add additional value sets.

   ii. Select the ***Constructor*** to be used from the pull-down list (in case multiple constructors are defined in the source code of your component), and specify the values for the constructor arguments as well. For our example you can use the following settings:
      - `0` for *ca_int*
      - `true` for *ca_bool*

   iii. Click the ***Finish*** button.

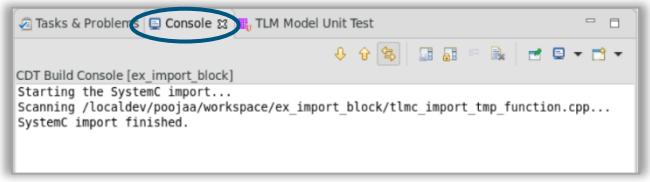
   The SystemC import is now finished. The ***Model Instance*** field is now populated automatically by the SystemC importer.

Note: This is the instance specification including namespaces, model name, template parameters and constructor arguments. You can modify this field if required and click ***Synchronize*** to re-synchronize the model.



You need to ***Synchronize*** every time you change your SystemC source code description or ***Build Configurations*** settings.

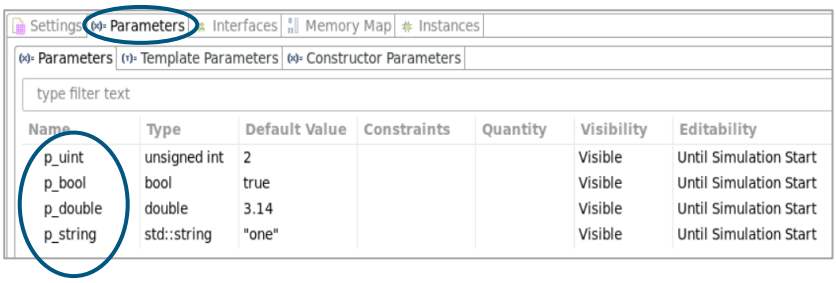
Use ***Import*** when you want to use the explicit dialog box to specify different parameters or constructor.



After the import some fields become read-only. The reason is that the model reflects the SystemC source code as it is. TLM Creator does not re-generate the source code based on settings in the TLM Creator project. However, several fields can still be edited. These are usually the metadata that is stored within the TLM Creator model, and then passed on to tools like VDK Creator and Platform Creator.

Let's now inspect the other tabs.

9. Click to open the ***Parameters*** tab.



All *scml_properties* are imported as parameters with the same name as given in the *scml_property* constructor.

```
sc_module(name)
, in("in")
, socket("socket")
, p_reset("p_reset")
, p_clk("p_clk")
, m_ui("p_uint", 2)
, m_b("p_bool", true)
, m_d("p_double", 3.14)
, m_s("p_string", "one")
, MemoryMap("MemoryMap", (12) / scml2::sizeOf<unsigned int >())
```

```
scml_property<unsigned int> m_ui;
scml_property<bool> m_b;
scml_property<double> m_d;
scml_property<std::string> m_s;
```

Note that you cannot add, remove, rename parameters, or change their type.

However, you can change some details of parameter such as, their ***Default Value***, ***Constraints***, ***Visibility***, ***Value*** , ***Editability***, and ***Description***. This metadata information is then packaged into the TLM Creator model and used in tools like VDK Creator (inside Virtualizer Studio) or Platform Creator.

Review the parameters specifications in the snippet of the code from the imported block. Same values are imported in the ***Parameters*** tab.

10. In the same way as for the SCML Parameters, the ***Template Parameters*** and ***Constructor Parameters*** (all in the ***Parameters*** page) cannot be added, removed, or renamed, but you are able to change the metadata information.

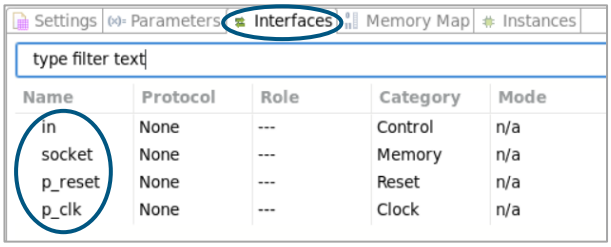
Review the ***Template Parameters*** view and the ***Constructor Parameters*** to see which fields are (non-)editable and how they relate to the SystemC source code of the model.

11. Click to open the ***Interfaces*** tab.

All ports and sockets are imported. You cannot add, remove, or rename interfaces nor can you change their ***Type*** or ***Quantity***.

You can specify the metadata information like ***Category***, ***Active Level*** for ***Reset*** ports, and ***Description***. This information is then passed to TLM Creator model and used in tools like VDK Creator or Platform Creator. The ***Category*** is preselected based on the interface name and type.

Review the interfaces specifications in the snippet of the code from the imported block. Same values are imported in the ***Interfaces*** tab.




```
template<typename Ttn, bool Tb, int Tint>
class ex_import_block : public sc_module {
public:
  sc_in<bool> in;
  tlm::tlm_target_socket<32> socket;
  sc_core::sc_in<bool> p_reset;
  sc_core::sc_in<bool> p_clk;

  SC_HAS_PROCESS(ex_import_block);
```

12. Next, open the ***Memory Map*** tab. Memories, registers, and bit-fields are **<u>not</u>** imported. You need to add the memories associated to target sockets in this tab manually so that at the time of export, the right target locations are created. Target locations is metadata that is passed to VDK Creator or Platform Creator for use during the VDK assembly page, indicating the size of the memories behind the target socket.

Note also that adding registers, bit-fields, and any callbacks is disabled in the TLM Creator GUI for imported SystemC models, as source code generation for imported SystemC models is not supported.

Review the memory and port adaptor information in the snippet of the code from the imported block.

Let's add this information in the ***Registers*** tab with related values. Click the ***Add Memory*** button and add the following details.

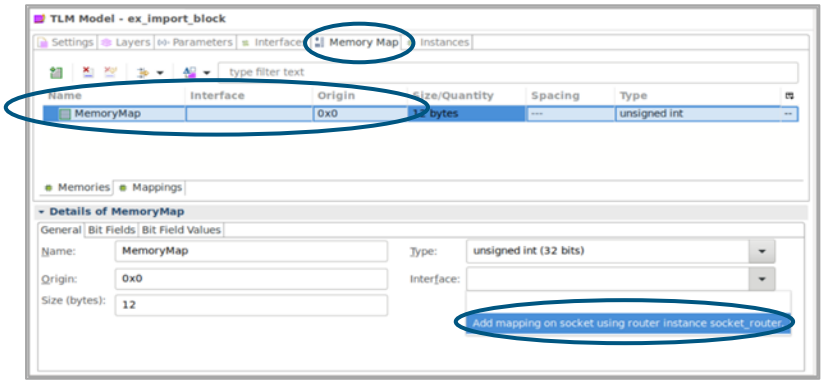
- `MemoryMap` in the ***Name*** field
- `0x0` in the ***Origin*** field,
- `12 bytes` in the ***Size/Quantity*** field,
- `unsigned int` in the ***Type*** field, and
- select the `Add mapping on socket using router instance` option in the ***Interface*** field.

```
  , MemoryMap("MemoryMap", (12) / scml2::sizeOf<unsigned int >())
{
  sensitive<<in;
  SC_METHOD(init);

  /* initialize adapter */
  socket_adapter = scml2::target_port_adaptor::create("socket_adapter", &socket);
  /* bind FT adapter to memory */
  (*socket_adapter)(MemoryMap);

  ...

  scml2::memory< unsigned int > MemoryMap;
  scml2::target_port_adaptor* socket_adapter;
};
```



Details can be viewed in the ***Mappings*** tab of the Details view.

- Save the TLM Creator project using the ***Save*** button ( ) or the keyboard shortcut `<Ctrl+S>`.

Our imported component is now ready for the next step.

Task-4: Build TLM Creator Model

Motivation: At this point, you can build the model. It will be compiled and built as a FastBuild library, which is a productivity technology used in Virtualizer Studio: each TLM model is compiled as a dynamic link library, which avoids the re-compilation & re-linking of netlist code, as the dynamic libraries of all netlist components are bound together at initialization time, and not statically by means of SystemC netlist code.

13. For enabling FastBuild, you need to add all the instances of the template parameters in the ***Build*** page of the ***Settings*** tab as follows.

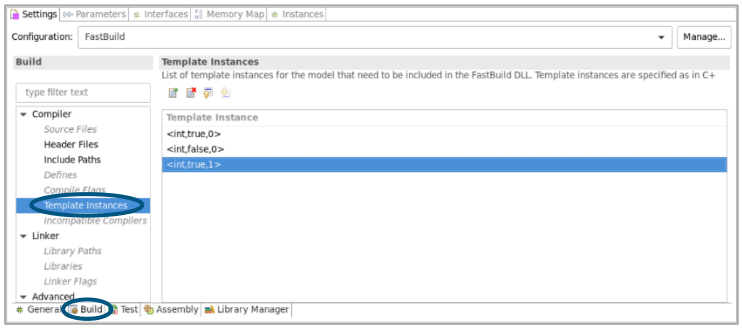
    i. Open the ***Build*** page.

    ii. Open the ***Template Instances*** settings under the ***Compiler*** head option in the left pane.

    iii. Next, add a new template parameter set to complement the version added during the early import stages. Remember that Virtualizer FastBuild will prebuild a DLL for each value set.

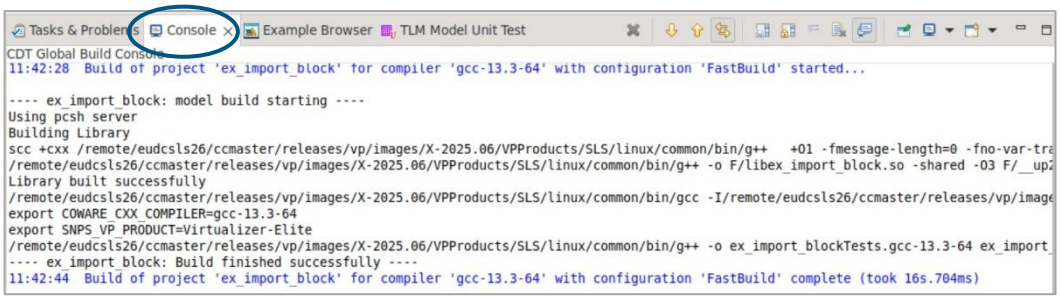
    Click ***Add Item*** and type the following template values separated by commas without any spaces:

    - `<int,true,0>`
    - `<int,false,0>`
    - `<int,true,1>`



14. Save your changes using the ***Save*** button ( ) or the keyboard shortcut `<Ctrl+S>` and click the ***Build*** button to build the component.

    The project gets saved, SystemC code gets compiled and all the template instances as specified above are built. If there are no errors, the library gets created.



The library is ready to be used in VDK Creator or Platform Creator.

Task-5: Executing Unit Tests

Motivation: A unit test framework is also automatically generated for the imported SystemC model. A default test is created to verify that the imported SystemC TLM model gets compiled and executed.

15. Select the `ex_import_block` project in the ***Project Explorer*** and run the unit tests using one of the following methods:

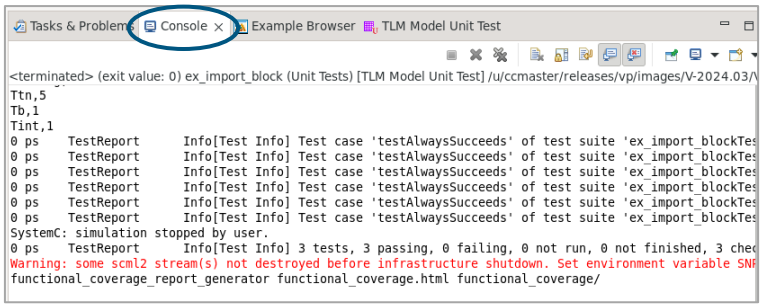
    - Left-click ***Run*** toolbar button ( ) or
    - Select ***Run → Run As → TLM Model Unit Test*** menu option.

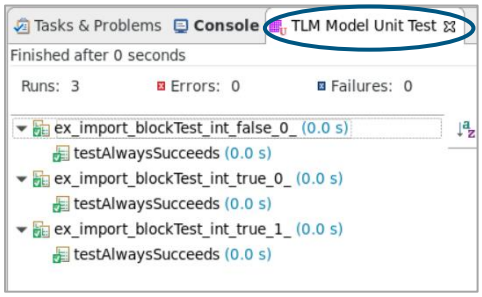
    The ***TLM Model Unit Test*** displays the status of the tests ran on the model graphically. It reports the number of tests, whether the test passed or failed, errors with an easy navigation to the source of the errors. Double-clicking an error takes you directly to the source code location of the failing test assertion.

    In this case, it shows the `testAlwaysSucceeds`, which is the default test, is passing successfully.

    Note that the test runner runs the test for each specified template parameter combination. Right-clicking on any test, allows you to selectively run it.

    The ***Console*** view displays the build messages and unit tests, which are helpful in debugging. It also shows the unit test statistics in the bottom.





16. To complete the model definition, you can add, execute your own unit tests, and view functional coverage results as described in the Lab Module 2 of this training program.

What We Learnt

In this module, we learned how to import a s SystemC component. We performed the following tasks:

- Create a new TLM project using the `systemc_import` template.
- Specify the SystemC model to be imported, i.e. source files and the model instance.
- Import the model into your project.
    - Review the imported model details in the ***Parameters***, ***Interfaces***, and ***Memory Map*** tabs.
- Build the TLM Creator model and run the default unit test.